

collection: "NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA" volume: 1 roman: "I" title: "MCP — A Língua Franca dos Agentes" subtitle: "Model Context Protocol: como agentes acessam ferramentas, dados e contexto de forma padronizada." edition: "Edição Canônica 1.0.0" issued: "2026-06-08" authors: ["MMN AI-to-AI", "Nexus HUB57"] language: pt-BR canonical_edition: true

NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA



Volume I — MCP — A Língua Franca dos Agentes

Model Context Protocol: como agentes acessam ferramentas, dados e contexto de forma padronizada.

Edição Canônica 1.0.0 · 2026-06-08 · MMN AI-to-AI · Nexus HUB57

Pergunta-âncora: Como agentes leem o mundo sem reinventar adaptadores a cada integração?

Sumário

1. O problema dos $N \times M$ conectores
2. Anatomia do MCP: cliente, servidor e host
3. As três primitivas: Resources, Tools, Prompts
4. Capabilities e o handshake de inicialização
5. Transporte: stdio, HTTP+SSE e o futuro WebSocket
6. Como construir um servidor MCP do zero
7. Sampling reverso: quando o servidor pede ao modelo
8. Segurança, escopo e o problema da confused deputy
9. MCP em produção: versionamento, telemetria, cache
10. Manifesto: por que o MCP venceu antes mesmo de vencer

1. O problema dos $N \times M$ conectores

Antes do MCP, integrar um LLM ao mundo era um problema combinatório. Cada modelo (Claude, GPT, Llama, Gemini, Mistral) precisava de um conector próprio para cada fonte (Slack, GitHub, Notion, Postgres, Jira, Drive, S3). Com N modelos e M fontes, a conta dava $N \times M$ integrações — todas com semânticas ligeiramente diferentes, todas mantidas por times distintos, todas quebrando em sincronia diferente quando a API do destino mudava.

O MCP, anunciado pela Anthropic em novembro de 2024 e rapidamente adotado por OpenAI, Google DeepMind, IBM e dezenas de fornecedores, resolve isso transformando o problema $N \times M$ em $N+M$: o modelo fala MCP, a fonte fala MCP, e o protocolo é o único ponto de tradução. A analogia honesta é o USB-C: não é o cabo mais elegante do mundo, é o cabo que **todos concordaram em usar**.

Essa concordância tem três consequências práticas que importam:

1. **Conectores viram commodity** — você não escreve mais um cliente Slack para cada modelo; escreve **um servidor MCP do Slack** e qualquer host MCP consome.
2. **Capacidades viram declarativas** — o agente descobre o que pode fazer ao se conectar, em vez de o desenvolvedor injetar a lista de tools no system prompt.
3. **Substituir o modelo deixa de quebrar o stack** — trocar Claude por GPT-5 exige zero alteração no servidor; o contrato é o protocolo, não o modelo.

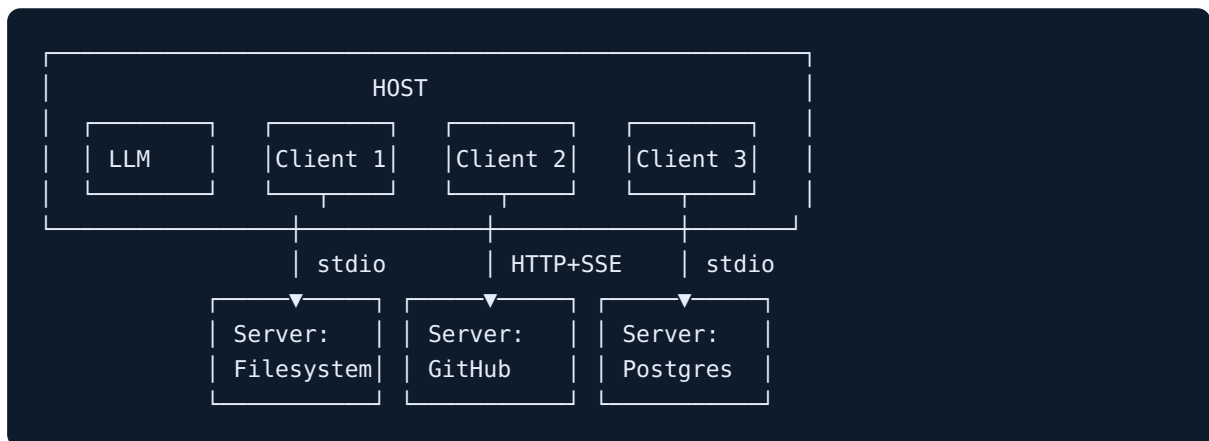
A primeira lição do MCP, portanto, não é técnica — é estratégica. Adotar MCP é admitir que **o valor a longo prazo está nos servidores, não nos modelos**.

2. Anatomia do MCP: cliente, servidor e host

A arquitetura do MCP tem três papéis, e confundi-los é a causa de 80% dos bugs iniciais:

- **Host** — a aplicação que o usuário enxerga (Claude Desktop, Cursor, Zed, um agente custom). O host **gerencia LLM e UI**.
- **Cliente** — a parte do host que **fala MCP**. Cada cliente mantém uma conexão 1:1 com um servidor. Um host pode ter N clientes simultâneos.
- **Servidor** — processo independente que **expõe** Resources, Tools e Prompts. Pode rodar local (stdio) ou remoto (HTTP+SSE).

A topologia canônica é uma estrela: o host no centro, vários servidores nas pontas, um cliente por servidor. O servidor **nunca fala diretamente com o LLM** — ele fala com o cliente, que fala com o host, que decide o que mostrar ao modelo.



Essa separação não é decorativa: ela permite que o **servidor seja inspecionável e auditável** sem privilégios sobre o modelo. O servidor é dumb-by-design — ele apenas responde a JSON-RPC. Toda inteligência fica no host.

3. As três primitivas: Resources, Tools, Prompts

MCP expõe exatamente três tipos de primitivas. Conhecer a diferença entre elas é o que separa um servidor bem-desenhado de uma bagunça.

Resources — dados contextuais (READ)

Resources representam **conteúdo legível** que o host pode anexar ao contexto do modelo. Cada resource tem URI (`file:///projeto/README.md`, `postgres://db/schema/users`, `slack://channel/general`), mimeType e conteúdo. São **idempotentes, sem efeito colateral**, e o host decide quando ler.

Regra prática: se a operação é "GET", é Resource.

Tools — ações com efeito (WRITE/EXEC)

Tools são funções com efeito colateral que **o modelo decide invocar**. Cada tool tem nome, descrição, input schema (JSON Schema) e retorna conteúdo estruturado. São o equivalente MCP de "function calling", mas com contrato versionável e auditável fora do prompt.

Regra prática: se a operação pode mudar o mundo, é Tool.

Prompts — templates reutilizáveis

Prompts são **workflows nomeados** que o servidor expõe. O usuário (não o modelo) seleciona um prompt; o servidor retorna mensagens estruturadas com argumentos preenchidos. São o ponto fraco mais subestimado do MCP: bem usados, eles **eliminam a engenharia de prompt do client-side**.

Regra prática: se um humano vai escolher antes do modelo agir, é Prompt.

A confusão clássica é expor como Tool algo que deveria ser Resource (porque "é mais fácil chamar"). Resultado: o modelo gasta tokens decidindo invocar algo que o host poderia ter anexado de graça.

4. Capabilities e o handshake de inicialização

Toda sessão MCP começa com um **handshake** que declara capabilities — quem suporta o quê. Esse handshake é o documento mais subestimado do protocolo.

```
// 1. Cliente → Servidor
{
  "jsonrpc": "2.0", "id": 1, "method": "initialize",
  "params": {
    "protocolVersion": "2025-03-26",
    "capabilities": {
      "roots": { "listChanged": true },
      "sampling": {}
    },
    "clientInfo": { "name": "claude-desktop", "version": "1.4.0" }
  }
}

// 2. Servidor → Cliente
{
  "jsonrpc": "2.0", "id": 1, "result": {
    "protocolVersion": "2025-03-26",
    "capabilities": {
      "resources": { "subscribe": true, "listChanged": true },
      "tools": { "listChanged": true },
      "prompts": { "listChanged": true },
      "logging": {}
    },
    "serverInfo": { "name": "github-mcp", "version": "0.3.2" }
  }
}

// 3. Cliente → Servidor (confirma)
{ "jsonrpc": "2.0", "method": "notifications/initialized" }
```

Três decisões críticas saem desse handshake:

- **protocolVersion** define o vocabulário. Se cliente e servidor divergem, ambos devem degradar para a versão menor — não falhar.
- **listChanged** declara se o servidor pode **notificar** mudanças (resource adicionado, tool removida). Sem ele, o cliente precisa fazer polling.
- **sampling** declara se o cliente expõe o LLM ao servidor para chamadas reversas (ver Capítulo 7).

Falhar nesse handshake silenciosamente é a causa #1 de "MCP que não funciona": o cliente assume capabilities que o servidor não anunciou.

5. Transporte: stdio, HTTP+SSE e o futuro WebSocket

MCP é agnóstico de transporte. Hoje três transportes dominam:

Transporte	Quando usar	Trade-off
stdio		

Transporte	Quando usar	Trade-off
	Servidor local, mesmo processo do host	Zero overhead, isolamento por processo, sem rede
HTTP + SSE	Servidor remoto, multi-tenant	Streaming via SSE, requer auth, latência maior
Streamable HTTP (2025)	Híbrido novo, single endpoint	Substitui SSE em deploys modernos

A escolha de transporte determina o modelo de segurança:

- **stdio** → segurança é o sandbox do processo (filesystem, env vars).
- **HTTP** → segurança é OAuth 2.1 + escopos por sessão.

Tentar fazer um servidor stdio rodar remoto via TCP é um anti-padrão recorrente e perigoso: você expõe stdio (sem auth) a uma rede pública. Não faça.

6. Como construir um servidor MCP do zero

O caminho mais curto entre "tenho uma API REST" e "tenho um servidor MCP" é mais curto do que parece. Esqueleto mínimo em Python (SDK oficial):

```

from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent
import asyncio, httpx

server = Server("clima-mcp")

@server.list_tools()
async def list_tools() -> list[Tool]:
    return [Tool(
        name="get_weather",
        description="Retorna o clima atual de uma cidade.",
        inputSchema={
            "type": "object",
            "properties": {"city": {"type": "string"}},
            "required": ["city"],
        },
    )]

@server.call_tool()
async def call_tool(name: str, arguments: dict) -> list[TextContent]:
    if name != "get_weather":
        raise ValueError(f"Tool desconhecida: {name}")
    async with httpx.AsyncClient() as c:
        r = await c.get(f"https://api.weather.example/{arguments['city']}")
    return [TextContent(type="text", text=r.text)]

async def main():
    async with stdio_server() as (read, write):
        await server.run(read, write, server.create_initialization_options())

if __name__ == "__main__":
    asyncio.run(main())

```

Cinco regras canônicas para servidores MCP em produção:

1. **Descrições de tools são parte da API.** O modelo decide invocar com base na descrição. Trate-as como documentação versionada, não como comentário.
2. **Input schema é contrato.** Use JSON Schema strict; rejeite extras com `additionalProperties: false`.
3. **Erros são dados, não exceções.** Retorne erro estruturado, não exception stack-trace.
4. **Cada chamada é idempotente onde for possível.** Idempotência permite retry seguro do host.
5. **Logs estruturados via `notifications/logging`**, não stderr cego.

7. Sampling reverso: quando o servidor pede ao modelo

A primitiva mais subestimada do MCP é **sampling**: o servidor pode pedir ao host que execute uma chamada de LLM em seu nome. Isso transforma servidores em agentes auxiliares sem que eles precisem ter credenciais de modelo.

```
# servidor solicita ao host: "rode o modelo com esses parâmetros"
result = await ctx.session.create_message(
    messages=[
        SamplingMessage(role="user", content=TextContent(
            type="text", text=f"Resuma este PR em 3 bullets:\n{pr_body}"
        ))
    ],
    max_tokens=500,
)
```

Por que isso importa: permite que **servidores MCP sejam compostáveis** sem duplicar chaves de API, sem decidir qual modelo usar, sem custo separado. O host mantém controle de gastos e política; o servidor ganha inteligência.

Cuidado canônico: **sampling exige consentimento explícito do usuário**. Não é uma porta dos fundos para o servidor consumir tokens do host sem aviso.

8. Segurança, escopo e o problema da confused deputy

MCP herda do mundo OAuth o pesadelo do **confused deputy**: um servidor com permissão ampla executa uma ação **em nome do usuário** com base em uma instrução **vinda do modelo**, que pode ter sido envenenada via prompt injection em um Resource lido anteriormente.

Mitigações canônicas, em ordem de eficácia:

1. **Princípio do menor escopo no token OAuth do servidor**. Não dê acesso `repo` quando `public_repo` resolve.
2. **Tool gating por confirmação humana** para ações destrutivas (`delete_*`, `transfer_*`, `pay_*`). O host deve abrir um modal, não confiar no modelo.
3. **Resources são potencialmente hostis**. Sanitize antes de injetar em prompts; trate o conteúdo como dados, não como instruções.
4. **Audit log fora do host**. Toda chamada de tool gera um evento exportável.

A regra de ouro: **um servidor MCP é tão seguro quanto o pior Resource que ele expõe**. Conteúdo lido vira prompt; prompt vira ação.

9. MCP em produção: versionamento, telemetria, cache

Levar MCP de demo para produção exige operacionalizar quatro pontos:

Versionamento. O campo `protocolVersion` é semver simplificado. Mantenha servidores no `latest - 1` por pelo menos 90 dias antes de subir; clientes em produção raramente acompanham o cutting edge.

Telemetria. Cada chamada MCP carrega um trace ID. Exporte para OpenTelemetry: spans por tool call, atributos com nome da tool, latência, status, tamanho do input/output. Isso permite responder à pergunta "qual tool está custando mais tokens ao modelo?".

Cache de Resources. Resources frequentes (schemas, configs, READMEs) devem ter ETag e `Cache-Control` no servidor; o cliente respeita. Sem isso, cada sessão recarrega o universo.

Health checks. Servidores HTTP expõem `/healthz` separado do protocolo MCP. Não use `initialize` como ping — é caro.

10. Manifesto: por que o MCP venceu antes mesmo de vencer

MCP venceu antes de ter a melhor implementação, antes de ter o melhor SDK e antes de cobrir todos os casos. Venceu pelo motivo mais antigo da engenharia: **foi o primeiro padrão que os concorrentes adotaram em vez de combater.**

A próxima geração de agentes não vai competir em "quem tem mais integrações nativas". Vai competir em **quem expõe os melhores servidores MCP** e em **quem tem o melhor host para consumi-los**. A camada de modelo virou commodity; a camada de protocolo virou o terreno disputado.

Tese final do volume: Quem ainda escreve conectores ad-hoc em 2026 está pagando o preço de uma decisão arquitetural que o mercado já tomou. MCP não é a melhor solução possível — é a solução que **virou inevitável**.

Checklist Canônico — MCP em produção

- [] Cada servidor expõe `serverInfo` com nome e versão semver.
- [] Handshake declara apenas capabilities realmente implementadas.
- [] Tools destrutivas exigem confirmação humana no host.
- [] Input schemas usam `additionalProperties: false`.
- [] Descrições de tools revisadas como documentação versionada.

- [] Resources frequentes têm ETag e Cache-Control.
- [] OAuth scopes seguem o princípio do menor privilégio.
- [] Traces OTel exportados para cada tool call.
- [] Política de versionamento (latest-1) documentada.
- [] Plano de revogação de token testado em ambiente real.

Glossário do Volume

- **Host** — aplicação dona da UI e do LLM (ex.: Claude Desktop).
- **Cliente** — componente do host que mantém uma conexão MCP 1:1.
- **Servidor MCP** — processo que expõe Resources/Tools/Prompts via JSON-RPC.
- **Resource** — dado contextual legível, sem efeito colateral.
- **Tool** — ação invocável pelo modelo, com efeito colateral declarado.
- **Prompt (MCP)** — template de workflow nomeado, selecionado pelo usuário.
- **Sampling** — primitiva que permite ao servidor pedir uma chamada de LLM ao host.
- **stdio** — transporte local via stdin/stdout do processo servidor.
- **Confused Deputy** — falha de segurança onde um componente privilegiado age sob instrução não autorizada.

Gancho para o Próximo Volume

MCP resolve **agente** ↔ **mundo**. Mas e quando dois agentes precisam falar entre si, descobrir capacidades um do outro, dividir tarefas e negociar resultados? Esse é o território do **Volume II — A2A: Comunicação entre Agentes**, onde o Agent-to-Agent protocol da Google entra em cena.